SYSTEM DESIGN

B-Trees & Indexing Visual Guide

The data structure behind every database index: wide, shallow, and always balanced.

ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Why Not Binary Trees

Disks read in PAGES (4-16KB). A binary tree wastes 99% of each page.

Binary tree

1 key per node■30 levels for 1B keys■30 disk reads

B-tree

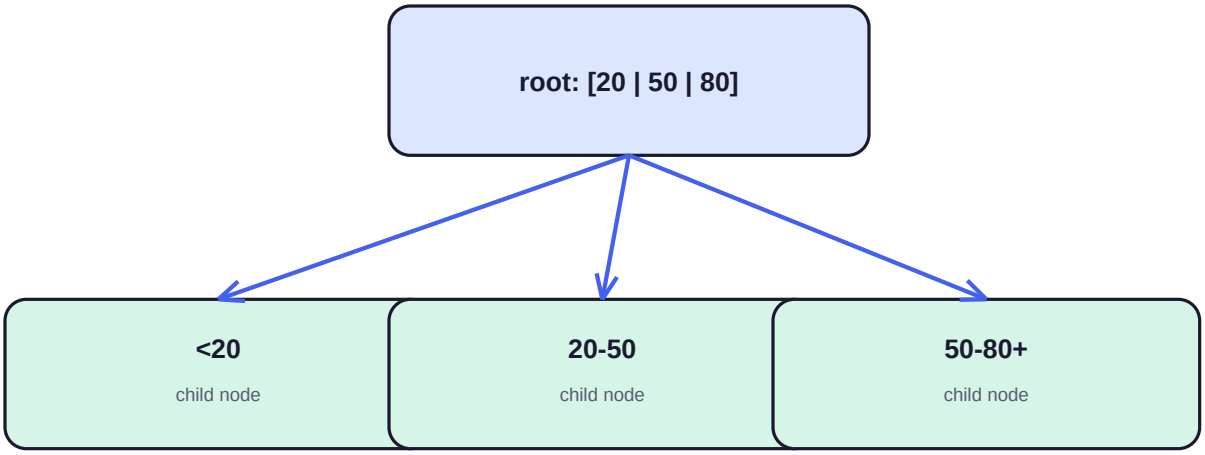
100s of keys per node■3-4 levels for 1B keys■3-4 disk reads

Design for the STORAGE medium: one node = one page = one I/O. Fan-out is everything.

STEP 2/6

Balanced Multi-Way Trees

Hundreds of keys per node, children between keys -- height stays tiny.



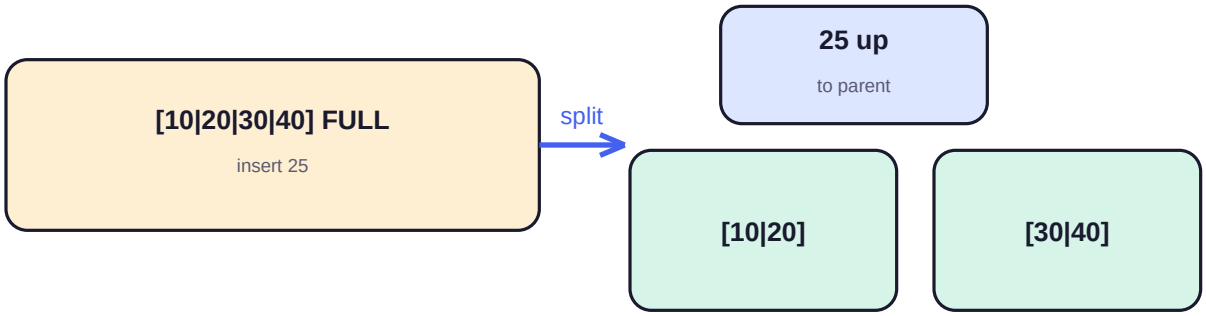
Search: compare within the node, descend to the child between the keys. O(log n) page reads.

Branching factor 100-1000: height = log_base(keys). A billion rows fits in 3-4 levels.

STEP 3/6

Node Splits Keep It Balanced

Full node? Split in two, push the middle key up -- the tree grows at the root.

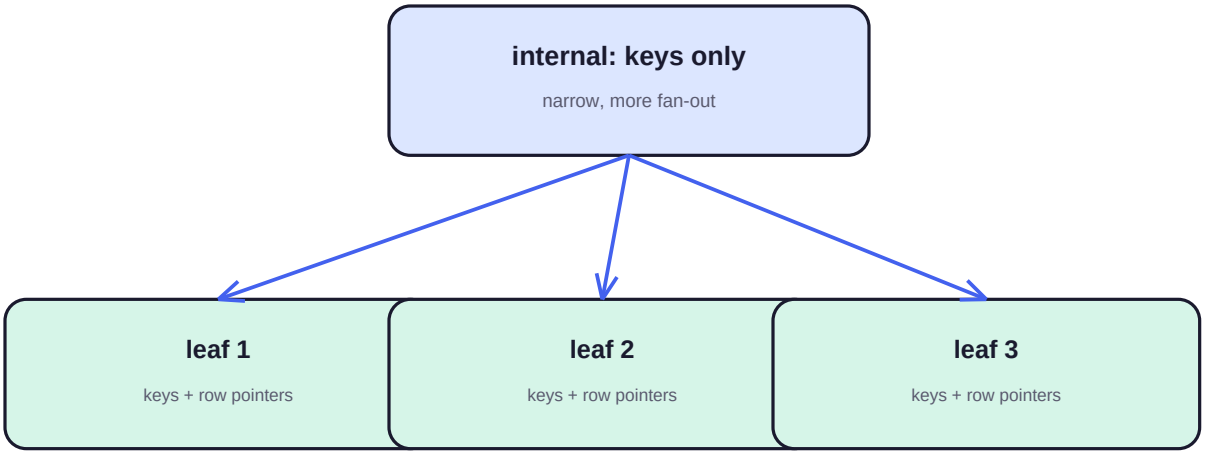


- Splits propagate upward only when parents are full too -- amortized, rare at the top.
- The tree stays PERFECTLY balanced: every leaf is the same depth, so worst case = average case.
- Underflow on delete merges siblings (the reverse) -- same invariant, opposite direction.

STEP 4/6

B+ Trees: Data in the Leaves

Internal nodes hold KEYS ONLY; all values live in a linked list of leaves.



Range scan (WHERE age BETWEEN 20 AND 30): find the start leaf, then walk the linked list.

This is why databases use B+ trees, not plain B-trees: sequential scans never climb back up.

STEP 5/6

Why Databases Use Them

Predictable I/O for point lookups AND range scans, with ordered iteration.

- Logarithmic reads AND writes: 3-4 page I/Os for a billion-row table, worst case.
- Order preserved: ORDER BY, BETWEEN, prefix LIKE 'abc%' all walk the leaf chain.
- Self-maintaining: splits/merges keep balance without a DBA rebuilding anything.
- Page-aligned: nodes sized to disk/buffer-pool pages -- the cache loves them.

STEP 6/6

The Costs

No free lunch: writes amplify and pages fragment.

- Write amplification: one row update rewrites a whole page (and the WAL, and maybe splits).
- Random inserts fragment pages (half-empty leaves) -- REINDEX/VACUUM exists for a reason.
- For write-mostly workloads, LSM trees (previous guide) trade read cost for write speed.
- Choose B+ tree when: reads dominate, range queries matter, data fits the update pattern.

STEP ✓/✓

Key Takeaways

B-Trees & Indexing — Remember

- One node = one disk page: fan-out of hundreds keeps billion-row trees 3-4 levels deep.
- Full nodes split and push the middle key up -- the tree stays perfectly balanced forever.
- B+ trees keep values only in leaves linked together: range scans walk the list, never climb.
- Logarithmic point lookups, ordered iteration, self-maintenance -- the database default.
- Costs: page-sized write amplification and fragmentation; LSM trees win write-heavy cases.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev